

Reporte técnico: Implementación y análisis de un demonio de monitoreo en Linux

1. Introducción

En esta práctica se implementó un demonio de monitoreo en un sistema operativo Linux utilizando un script desarrollado en Bash y administrado mediante systemd. El propósito fue observar el comportamiento del sistema operativo frente a un proceso que permanece activo en segundo plano y que realiza tareas periódicas de monitoreo.

El demonio fue configurado para registrar cada cinco segundos la fecha y hora, el número total de procesos activos y la memoria RAM disponible. Además, se configuró un mecanismo de recuperación automática para comprobar el comportamiento de systemd cuando el proceso es terminado de manera forzada mediante kill -9.

Esta práctica permitió relacionar conceptos teóricos de sistemas operativos, como procesos, memoria, ejecución en segundo plano, administración de servicios y recuperación de procesos, con su funcionamiento real dentro de Linux.

2. Objetivos

Objetivo general

Implementar y analizar un demonio de monitoreo en Linux administrado por systemd, observando su comportamiento como proceso persistente y su capacidad de recuperación automática.

Objetivos específicos

- Crear un script funcional utilizando Bash.
- Obtener información relacionada con los procesos y la memoria del sistema.
- Registrar periódicamente la información obtenida en una bitácora.
- Configurar el script como un servicio de systemd.
- Ejecutar el servicio en segundo plano.
- Analizar el comportamiento del servicio cuando su proceso es terminado mediante kill -9.
- Comprobar la capacidad de systemd para reiniciar automáticamente el servicio.

3. Descripción de la implementación

El monitoreo se realizó mediante un script Bash llamado monitor_linux.sh. El programa utiliza

un ciclo `while true`, por lo que permanece ejecutándose continuamente mientras el servicio esté activo.

En cada iteración se obtiene la fecha y hora mediante el comando `date`. Posteriormente, se utiliza `ps -e` para obtener información de los procesos existentes y `wc -l` para realizar el conteo correspondiente.

La memoria RAM disponible se obtiene mediante el comando `free`, utilizando `awk` para seleccionar el dato correspondiente.

Finalmente, los valores obtenidos se almacenan en el archivo `monitor_linux.log`. Después de cada registro, el comando `sleep 5` provoca una espera de cinco segundos antes de realizar la siguiente medición.

El script fue posteriormente registrado como un servicio mediante el archivo:

```
/etc/systemd/system/monitor-linux.service
```

La configuración incluyó:

```
Restart=always  
RestartSec=1
```

Estas instrucciones son fundamentales para la prueba de recuperación, ya que indican a `systemd` que debe intentar reiniciar el servicio cuando el proceso termine y esperar un segundo antes de hacerlo.

4. Análisis del comportamiento del sistema operativo

Durante la ejecución del demonio se observó que la cantidad de procesos activos y la memoria RAM disponible no permanecen constantes. Estos valores pueden cambiar incluso cuando el usuario no está ejecutando directamente nuevos programas.

Esto se debe a que un sistema operativo moderno mantiene numerosos procesos y servicios funcionando simultáneamente. Algunos pertenecen al usuario, mientras que otros corresponden al propio sistema, controladores, servicios de red, interfaz gráfica y diferentes componentes del entorno de escritorio.

Por esta razón, al realizar las mediciones cada cinco segundos es normal encontrar pequeñas variaciones en el número de procesos. Un cambio en este valor no necesariamente representa un problema, sino que demuestra que el sistema operativo administra dinámicamente sus procesos.

De manera similar, la memoria RAM disponible puede aumentar o disminuir durante la ejecución. Linux utiliza la memoria de forma dinámica y puede utilizar parte de ella como caché y buffers. Por lo tanto, la memoria disponible no debe interpretarse simplemente como memoria "vacía", sino como un recurso que el sistema administra de acuerdo con las necesidades de los procesos.

El registro periódico permitió observar este comportamiento dinámico sin necesidad de

modificar directamente los procesos del sistema.

5. Análisis del comportamiento de los procesos

Un proceso representa una instancia de un programa que se encuentra en ejecución. En esta práctica, el script de monitoreo se convirtió en uno de estos procesos.

Cuando el servicio fue iniciado mediante `systemctl start`, `systemd` creó y supervisó el proceso correspondiente al script. Este proceso recibió un PID, que es el identificador utilizado por Linux para distinguirlo de los demás procesos.

Una observación importante fue que el PID puede cambiar después de reiniciar el servicio. Esto ocurre porque cuando un proceso termina, el nuevo proceso creado posteriormente recibe un identificador de proceso diferente.

La prueba con `kill -9` permitió observar este comportamiento directamente. Al enviar la señal `SIGKILL`, el proceso fue terminado de manera inmediata. Sin embargo, el servicio continuó siendo administrado por `systemd`, que detectó la terminación y creó una nueva instancia debido a la configuración de reinicio automático.

6. Análisis del comportamiento de systemd

`systemd` funciona como el sistema de inicialización y administrador de servicios de muchas distribuciones modernas de Linux. Una de sus funciones es controlar el ciclo de vida de determinados procesos que son considerados servicios del sistema.

En esta práctica, `systemd` permitió iniciar, detener, consultar y reiniciar el demonio.

La instrucción:

```
Restart=always
```

fue especialmente importante. Esta configuración establece que el servicio debe ser reiniciado cuando termina, incluso si la terminación no fue causada por una salida normal del programa.

Por otra parte:

```
RestartSec=1
```

estableció un intervalo de un segundo antes del intento de reinicio.

La prueba realizada mediante `kill -9` permitió comprobar de manera práctica esta configuración. Aunque el proceso fue terminado forzosamente, `systemd` volvió a iniciar el servicio automáticamente.

Esto demuestra una diferencia importante entre el proceso y el servicio: el proceso puede morir, mientras que `systemd` puede mantener disponible el servicio creando una nueva instancia del proceso.

7. Justificación teórica de las observaciones

Las observaciones obtenidas durante la práctica pueden explicarse mediante los mecanismos básicos de administración de recursos de un sistema operativo.

En primer lugar, Linux utiliza un modelo basado en procesos. Cada programa en ejecución posee un contexto y un PID que permite al sistema operativo identificarlo y administrarlo.

En segundo lugar, el sistema operativo utiliza mecanismos de planificación para distribuir el tiempo de procesamiento del CPU entre los diferentes procesos. Por esta razón, existen múltiples procesos activos aunque el usuario no esté interactuando directamente con ellos.

En cuanto a la memoria, Linux administra la RAM entre los procesos y diferentes mecanismos internos como cachés y buffers. Por esta razón, el valor de memoria disponible puede cambiar constantemente.

Finalmente, systemd proporciona una capa de administración superior al proceso individual. El administrador de servicios puede detectar que un proceso terminó y ejecutar nuevamente el programa de acuerdo con las reglas establecidas en su archivo de servicio.

Por lo tanto, la recuperación observada después de utilizar `kill -9` no significa que el proceso original haya sobrevivido. El proceso original realmente terminó. Lo que ocurrió fue que systemd, al detectar su terminación, creó una nueva instancia del servicio.

8. Análisis reflexivo

La práctica permitió observar que un sistema operativo no es un entorno estático. Aunque el usuario no realice ninguna acción, Linux continúa ejecutando procesos y administrando recursos constantemente.

El monitoreo periódico hizo evidente que los valores de procesos y memoria cambian con el tiempo. Esto permite comprender que la administración de recursos es una actividad continua del sistema operativo.

También resultó importante observar que un proceso y un servicio no son exactamente lo mismo. El proceso corresponde a una ejecución concreta de un programa, mientras que el servicio representa una configuración que permite a systemd administrar esa ejecución.

La prueba con `kill -9` fue especialmente útil porque permitió comprobar esta diferencia de forma práctica. El proceso fue eliminado, pero el servicio permaneció disponible y fue iniciado nuevamente automáticamente.

Desde una perspectiva de administración de sistemas, esta característica es importante para servicios que deben permanecer disponibles. Un administrador puede establecer políticas de recuperación sin necesidad de mantener manualmente un proceso abierto o supervisarlo constantemente.

9. Conclusiones

La implementación permitió crear correctamente un demonio de monitoreo utilizando Bash y systemd. El programa logró registrar periódicamente información sobre procesos y memoria, manteniéndose activo en segundo plano.

El análisis permitió comprobar que los procesos y recursos del sistema operativo cambian constantemente debido a la actividad normal del sistema. También se comprobó que Linux administra dinámicamente la memoria y los procesos, incluso cuando el usuario no realiza acciones visibles.

La prueba de terminación mediante kill -9 permitió demostrar experimentalmente el funcionamiento de systemd como administrador de servicios. El proceso original fue terminado, pero systemd detectó la finalización y creó una nueva instancia gracias a las opciones de reinicio configuradas.

En conclusión, la práctica permitió relacionar conceptos teóricos de sistemas operativos con observaciones realizadas directamente en Linux. Se comprobó la importancia de los procesos, los PID, la administración de memoria y los mecanismos de supervisión y recuperación de servicios. Esto proporciona una base práctica para comprender cómo se administran procesos persistentes en sistemas Linux.